Reading the QueueLess Code

How to follow any feature from a click in the browser, through the API, into MongoDB and Redis, and back to every open screen.

For: Jayasri · Project folder: `~/Queueless` · Written 2 November 2026 · Line numbers match the code on that date (they shift a little as you edit; search for the function name if a number is off).

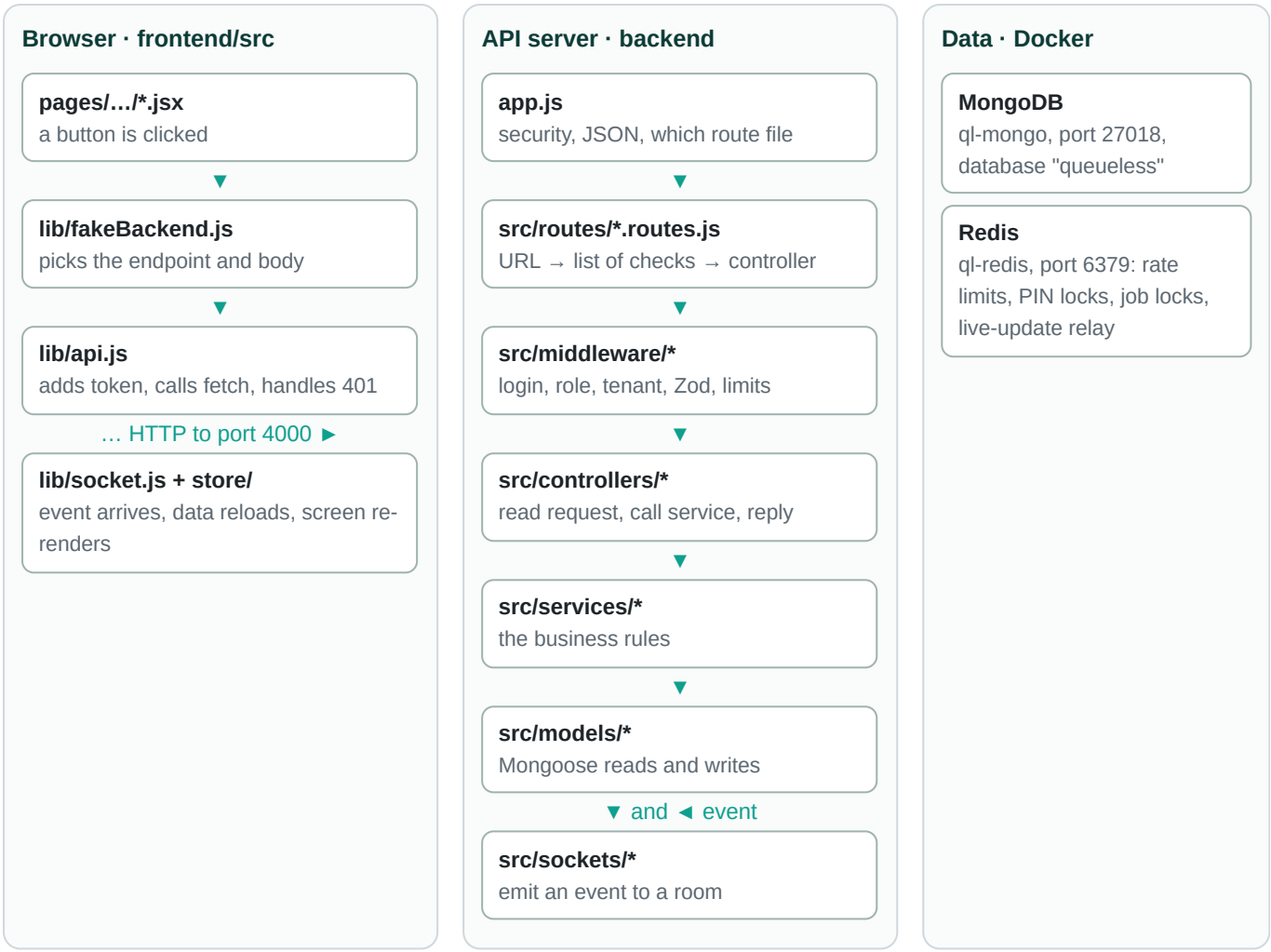
> **The one rule to remember.** Every feature is a chain of small files, and the chain is always in the same order: `page → fakeBackend.js → api.js → fetch → app.js → routes → middleware → controller → service → model → MongoDB`, then back up as a JSON answer, and sideways to other screens as a Socket.IO event. Learn the chain once and you can read any feature.

Contents

1. The big picture
2. Where each side starts
3. Walkthrough 1: logging in
4. Walkthrough 2: saving a service
5. Walkthrough 3: a customer joins the queue
6. Walkthrough 4: staff call next (live update)
7. Walkthrough 5: reloading the page
8. What each backend layer checks
9. Lookup tables: from what you see to the file
10. Tools for following a request
11. A reading plan for one week

1. The big picture

QueueLess has two programs. The **frontend** (React, in `frontend/src`) runs in the browser on port 5173. The **backend** (Express, in `backend`) runs on port 4000 and is the only part that touches the databases. They talk in two ways: normal HTTP requests (`fetch`) for asking and changing things, and a Socket.IO connection for live "something changed" messages.



The browser never talks to MongoDB or Redis directly. If you are looking for "where is the data saved", the answer is always a **service** calling a **model** on the backend.

2. Where each side starts

Frontend: from the browser opening the page

File · line	What happens there
`frontend/index.html`	The one HTML page. It loads `src/main.jsx`.
`src/main.jsx:11`	`restoreSession()`: asks the backend "is someone already logged in?" (using the refresh cookie). Then React draws `<App/>`.
`src/App.jsx:55`	`<Routes>`: the list of every page and its URL. Read this file first: it is the table of contents of the frontend.
`src/App.jsx:65, 75, 102`	Examples: `/login` → Login page; `/admin` → owner pages (wrapped in `RequireRole`); `/:slug` → a business's public join page.
`src/layouts/RequireRole.jsx:9`	The guard: waits for the session check, sends you to `/login` if you are not logged in, or to your own home if your role is wrong.
`src/pages/…`	One file per screen. This is where buttons and forms live.

Backend: from `npm run dev`

File · line	What happens there
`backend/server.js:13`	Connects MongoDB, then Redis.
`backend/server.js:20`	`initSockets(…)` : starts Socket.IO on the same port, with the Redis relay.
`backend/server.js:32`	`server.listen(4000)` : from now on requests arrive. Jobs and the email check also start here.
`backend/app.js:38–49`	Every request first passes security headers (helmet), CORS, cookies, and JSON parsing.
`backend/app.js:67`	The general rate limit for everything under `/api` .
`backend/app.js:70–80`	Which route file handles which URL prefix: `/api/auth` , `/api/admin` , `/api/public` , `/api/staff` , … This is the table of contents of the backend.
`backend/app.js:86`	`errorMiddleware` : the last stop. Any error anywhere becomes `{ ok:false, error:{ code, message } }` .

How to read a route line. Everything between the URL and the last function runs in order, left to right. For example `src/routes/auth.routes.js:11` :

```
router.post('/login', loginLimit, validate({ body: loginSchema }), auth.login);
//           URL        rate limit  check the body with Zod         controller
```

If any step says "no", the request stops there and the later ones never run.

3. Walkthrough 1: logging in

Goal: see how an email and password become a logged-in screen.

1. FRONTEND `pages/auth/Login.jsx:55` — the form's submit calls `login(email, password)` .
2. FRONTEND `lib/fakeBackend.js:185` `login()` — calls `api.post('/api/auth/login', { email, password })` .
3. FRONTEND `lib/api.js:64` `request()` → `fetch()` at **line 72** sends `POST http://localhost:4000/api/auth/login` with the JSON body.
4. BACKEND `app.js:70` sends `/api/auth/…` to `src/routes/auth.routes.js` .
5. BACKEND `auth.routes.js:11` — `loginLimit` (counts failed logins in Redis), then `validate(loginSchema)` (email format, password present).
6. BACKEND `controllers/auth.controller.js:39` `login()` — calls the service.
7. BACKEND `services/auth.service.js:46` `login()` — finds the user by email, compares the password with `passwordHash` using bcrypt, checks the account is active.
8. MONGODB Reads the `users` collection (model `src/models/User.js`).

9. BACKEND `auth.service.js:25` `issueTokens()` — makes the 15-minute access token and the 30-day refresh token. The controller sets the refresh token as the `ql_refresh` cookie and replies `{ ok:true, user, accessToken, home }`.

10. FRONTEND `fakeBackend.js:110` `begin()` — keeps the access token in memory, loads `/api/auth/me` (`loadSession`, line 83) into the stores, and opens Socket.IO (`startLive`, line 144). The Login page then navigates to `/admin` or `/staff`.

> **Try it:** open DevTools → Network, log in, click the `login` request. Compare its Payload and Response with steps 3 and 9. Then look at Application → Cookies for `ql_refresh`.

4. Walkthrough 2: saving a service

Goal: a typical "owner changes something" flow, including the checks that protect it.

1. FRONTEND `pages/admin/ServiceEdit.jsx:20` — Save calls `saveService(business.id, data, service?.id)`.

2. FRONTEND `lib/fakeBackend.js:270` — builds `{ name, avgMinutes, prefix, isActive, handledBy }`; new → `POST /api/admin/services`, existing → `PATCH /api/admin/services/:id`.

3. FRONTEND `lib/api.js:72` — `fetch` with header `Authorization: Bearer <access token>`.

4. BACKEND `app.js:75` → `src/routes/admin.routes.js`. At the top of that file one line protects every admin route: `requireAuth, requireOwner, tenantScope`.

5. BACKEND `middleware/auth.middleware.js:9` `requireAuth` — checks the token and loads the user. `requireOwner` — owners only. `middleware/tenant.middleware.js:12` `tenantScope` — loads **your** business into `req.business`; every later query uses it, so you can never touch another business.

6. BACKEND `admin.routes.js:32` — `validate(createServiceSchema)` checks the body (rules in `src/validators/business.validator.js`).

7. BACKEND `controllers/admin.controller.js:52` `createService()` → `services/catalog.service.js:77` `createService()`: saves the service, links the chosen doctors/counters, writes an activity entry.

8. MONGODB New document in `services`; `resources.serviceIds` updated; a line in `activitylogs`.

9. FRONTEND Back in `fakeBackend.js`, `loadSession()` reloads the business so the Services list shows the new row; the page shows "Service added".

5. Walkthrough 3: a customer joins the queue

Goal: the public side, where there is no login, and how a token number is made safely.

1. FRONTEND `layouts/CustomerLayout.jsx` loads the business for the URL's slug (hook `hooks/usePublicBusiness.js`, which calls `GET /api/public/:slug`).

2. FRONTEND `pages/customer/Join.jsx:67` — "Get my token" calls `joinQueue(…, { slug })`.
3. FRONTEND `lib/fakeBackend.js:328` `joinQueue()` → `POST /api/public/<slug>/tokens` with `{ customer: { name, phone, … }, serviceId or resourceId }`.
4. BACKEND `app.js:76` → `src/routes/token.routes.js:14`. The checks, in order: `joinIpLimit` (5 per device), `publicBusiness` (find the business by slug; an old slug answers with a redirect), `joinPhoneLimit` (3 per phone), `requireOpenQueue` (live and open), `validate(joinSchema)`.
5. BACKEND `controllers/token.controller.js:11` `join()` → `services/token.service.js:104` `joinQueue()`: checks the business's own join form, returns the existing token if this phone is already in line, enforces the daily limit, and picks the line.
6. BACKEND `utils/tokenNumber.js:22` `nextTokenNumber()` — one atomic `$inc` on today's counter in the `queues` collection, inside a transaction. That is why two customers can never get the same number.
7. MONGODB New document in `tokens` (status `WAITING`, `guestSecretHash`); `queues.counters` goes up by one.
8. LIVE `token.service.js:158` `emitQueueEvent('token:created', …)` → staff screens reload their lines; join pages and the TV get new counts.
9. FRONTEND The answer includes `guestSecret` once. `Join.jsx` saves it in localStorage (`hooks/useGuestToken.js`) and opens the token page.

6. Walkthrough 4: staff call next (and every screen updates)

Goal: the real-time path. One click changes the database, then three different screens update without reloading.

1. FRONTEND `components/staff/NowServingPanel.jsx:47` — the Call next button runs `actions.callNext(resource.id)`.
2. FRONTEND `hooks/useQueueActions.js:19` → `lib/fakeBackend.js:364` `callNext()` → `POST /api/staff/resources/:id/call-next`.
3. BACKEND `app.js:77` → `src/routes/staff.routes.js`: `requireAuth`, role must be owner/manager/staff, `tenantScope`; then line 25 → `controllers/staff.controller.js:24`.
4. BACKEND `services/staff.service.js:55` `callNext()` — finishes the current customer, then claims the next waiting one with a single `findOneAndUpdate` (line 64). Because it is one atomic step, two staff pressing at the same instant can never get the same customer.
5. MONGODB The token's `status` becomes `CALLED`, with `calledAt` and `resourceId`.
6. LIVE `staff.service.js:70–71` emits `token:updated` and `token:called`. `sockets/queue.socket.js:141` `emitQueueEvent` → `broadcast()` (line 101) decides which room gets what: staff room (full details), the customer's own token room, the TV room (code and counter only, line 117).
7. LIVE Redis relays the event, so screens connected to any server instance receive it.

8. FRONTEND **Customer's phone:** `pages/customer/TokenStatus.jsx:104–105` listens for `token:updated` / `token:called` (it joined its room at line 97 with the guest secret) and reloads its status → "It is your turn".

9. FRONTEND **TV:** `hooks/useDisplay.js:45` receives `token:called` → flash and chime, then reloads.

10. FRONTEND **Other staff and owner screens:** `lib/fakeBackend.js:156–167` listen for `queue:updated` / `token:updated` and call `loadQueue()` (line 339). Home, Customers and Analytics use `hooks/useLiveQuery.js:11`, which reloads on the same events.

> **Try it:** open the staff console in one tab and the customer's token page in another. In the token tab, DevTools → Network → filter `socket.io` → click the websocket row → Messages. Press Call next in the other tab and watch `42["token:called",…]` arrive.

7. Walkthrough 5: reloading the page

Goal: understand why you stay logged in after a refresh, even though the access token is only in memory.

1. FRONTEND `main.jsx:11` runs `restoreSession()` (`fakeBackend.js:103`) before any page needs data. Pages behind a login show a spinner meanwhile (`RequireRole.jsx`).

2. FRONTEND `lib/api.js:44` `refreshSession()` → `fetch` at line 45: `POST /api/auth/refresh`. The browser attaches the `ql_refresh` cookie by itself.

3. BACKEND `auth.service.js` `refresh()` checks the cookie's signature and that its version number still matches `users.refreshTokenVersion` (logout increases it, which kills old cookies).

4. FRONTEND A new access token comes back; `loadSession()` fills the stores; `startLive()` reconnects Socket.IO. No cookie, or an old one → you see the login page.

5. FRONTEND Later, any request that gets `401 TOKEN_EXPIRED` is refreshed and repeated automatically inside `api.js` `request()` (lines 82–85).

8. What each backend layer checks

Layer	Files	Stops the request with
Rate limits	`middleware/rateLimit.middleware.js`	429 Too many requests (counters in Redis `rl:*`)
Login	`middleware/auth.middleware.js`	401 UNAUTHORIZED / TOKEN_EXPIRED
Role	`middleware/role.middleware.js`	403 (e.g. staff calling an owner route)
Tenant	`middleware/tenant.middleware.js`	403 / 404 (another business's data)
Validation	`middleware/validate.middleware.js` + `src/validators/*`	400 VALIDATION_ERROR, with `details` per field

Public business	`middleware/tenant.middleware.js` (`publicBusiness`), `goLive.middleware.js`	404 unknown slug, 308 old slug, 409 not live / closed
Business rules	`src/services/*`	409 conflicts (already called, email taken), 400 rule broken
Errors	`middleware/error.middleware.js:15`	Turns every error into the same JSON shape; unexpected ones are 500 and logged with a stack trace

Plan limits (`middleware/planLimit.middleware.js`) are switched off (`PLAN_LIMITS=off`), so they never stop a request in your setup.

9. Lookup tables: from what you see to the file

"I clicked this on screen. Where is the code?"

Screen	Page file (frontend/src/pages)	Function in fakeBackend.js	Backend service
Log in / sign up	`auth/Login.jsx`, `auth/Signup.jsx`	`login`, `signup`	`auth.service.js`
Accept invite	`auth/AcceptInvite.jsx`	`acceptInvite`	`invite.service.js`
Reset password	`auth/ResetPassword.jsx`	`resetPassword`	`auth.service.js`
Admin Home, Analytics	`admin/Home.jsx`, `admin/Analytics.jsx`	`getAnalytics`, `getActivity`	`analytics.service.js`, `activity.service.js`
Services, doctors/counters	`admin/Services.jsx`, `ServiceEdit.jsx`, `Resources.jsx`, `ResourceEdit.jsx`	`saveService`, `saveResource`, …	`catalog.service.js`
Customers	`admin/Customers.jsx`	`getCustomers`	`customers.service.js`
Staff and invites	`admin/Staff.jsx`	`loadStaff`, `createInvite`, `updateStaff`	`invite.service.js`
Settings, link, logo	`admin/Settings.jsx` + `admin/settings/*Tab.jsx`	`saveSettings`, `updateBranding`, `changeSlug`	`business.service.js`, `branding.service.js`, `slug.service.js`
Go live, pause	`admin/Channels.jsx`	`goLive`, `setQueueOpen`	`business.service.js`
Staff console	`staff/Console.jsx` (+ `components/staff/*`)	`loadQueue`, `callNext`, `markDone`, …	`staff.service.js`, `token.service.js`
Switch user (PIN)	`staff/SwitchUser.jsx`	`switchUserWithPin`	`staff.service.js`

Join page, token page	`customer/Join.jsx`, `customer/TokenStatus.jsx`	`joinQueue`, `getTokenStatus`, `leaveQueue`, `rateToken`	`token.service.js`
TV display	`display/Display.jsx` (+ `hooks/useDisplay.js`)	`getDisplay`	`business.service.js` (`displayView`)
Contact sales	`marketing/ContactSales.jsx`	`sendEnterpriseLead`	`enterprise.service.js`

"I know the endpoint. Where does it go?"

URL starts with	Route file (backend/src/routes)	Controller (backend/src/controllers)
`/api/auth`	`auth.routes.js`	`auth.controller.js`
`/api/onboarding`	`onboarding.routes.js`	`onboarding.controller.js`
`/api/business`	`business.routes.js`	`business.controller.js`
`/api/admin/staff`, `/api/admin/invites`, `/api/invites`	`invite.routes.js`	`invite.controller.js`
`/api/admin` (everything else)	`admin.routes.js`	`admin.controller.js`
`/api/public`	`token.routes.js`	`token.controller.js`
`/api/staff`	`staff.routes.js`	`staff.controller.js`
`/api/display`	`display.routes.js`	`display.controller.js`
`/api/billing`	`billing.routes.js`	`billing.controller.js`
`/api/enterprise`	`enterprise.routes.js`	`enterprise.controller.js`

From the controller, the service it calls is named at the top of the file in its `import` lines.

"I see it in Compass. Which code writes it?"

Collection	Model	Written mainly by
`users`	`User.js`	`auth.service.js`, `invite.service.js`
`businesses`	`Business.js`	`auth.service.js` (signup), `business.service.js`, `branding.service.js`, `slug.service.js`
`services`, `resources`	`Service.js`, `Resource.js`	`catalog.service.js`, `staff.service.js` (pause)
`tokens`, `queues`	`Token.js`, `Queue.js`	`token.service.js`, `staff.service.js`, `utils/tokenNumber.js`, jobs
`staffinvites`	`StaffInvite.js`	`invite.service.js`
`activitylogs`, `notifications`	`ActivityLog.js`, `Notification.js`	`activity.service.js`, `notification.service.js`

10. Tools for following a request

Tool	How to use it to follow the code
DevTools → Network	Filter **Fetch/XHR**. Click a request: **Headers** (URL, method, `Authorization`), **Payload** (what the frontend sent), **Response** (what the backend answered). The URL tells you the route file (table above).
DevTools → Sources (breakpoint)	Open `src/lib/api.js`, click the line number of `request()` (line 64). Click any button: the browser pauses there, and the **Call Stack** panel shows exactly which page and function made the call.
`console.log`	Add `console.log(method, path, body)` at the top of `request()` to print every API call in the browser console. On the backend, add one at the top of a controller or service; it prints in the terminal running `npm run dev`. Remove them when done.
Backend terminal	Every request is printed (e.g. `POST /api/staff/resources/…/call-next 200 23 ms`). Crashes print a stack trace: the first line inside `backend/src/` is usually where to look.
VS Code: Go to Definition	Ctrl+click (or F12) on a function name, e.g. `saveService` in a page, jumps to `fakeBackend.js`. Ctrl+Shift+F searches the whole project, e.g. for `'/api/admin/services'` or `emitQueueEvent('token:called'`.
Postman	Call one endpoint directly, without the frontend, to check whether a bug is in the frontend or the backend.
MongoDB Compass	`mongodb://localhost:27018/?directConnection=true` → database `queueless`. Refresh a collection after a click to see what was written.
Redis	`docker exec -it ql-redis redis-cli`, then `SCAN 0`. Live events: `PSUBSCRIBE 'socket.io*'`.
Tests	`backend/tests/<area>.test.js` describe each feature's expected behaviour in plain sentences. Run one: `npm test -- tests/queue.test.js`.

A debugging habit that saves time. When something is wrong, first decide which side it is on. Look at the request in DevTools → Network. If the **Payload** is wrong, the bug is in the frontend (page or `fakeBackend.js`). If the payload is right but the **Response** is wrong or an error, the bug is in the backend: follow the URL to its route file and go down the chain. If the response is right but the screen is wrong, the bug is in the page or the store.

11. A reading plan for one week

Day	Read	Goal
1	`frontend/src/App.jsx`, `backend/app.js`, `backend/server.js`	Know every page and every URL prefix.
2	Walkthrough 1 (login) and 5 (reload): `Login.jsx`, `fakeBackend.js` (session part), `api.js`, `auth.*` files	Explain access token, refresh cookie and logout in your own words.

3	Walkthrough 2 (service): `admin.routes.js`, all of `src/middleware/`, `catalog.service.js`	Name every check a request passes and the error each gives.
4	Walkthrough 3 (join): `token.routes.js`, `token.service.js`, `tokenNumber.js`, `Token.js`	Explain why two customers never get the same number.
5	Walkthrough 4 (call next): `staff.service.js`, `sockets/queue.socket.js`, `rooms.js`, `TokenStatus.jsx`, `useDisplay.js`	Draw who receives which event, and why customer names never reach the TV.
6	`backend/tests/queue.test.js` and `tenantIsolation.test.js`	See the rules from the outside; change one rule and watch a test fail.
7	Pick any button in the app and trace it alone, using only this guide's lookup tables.	You can find any feature's code in under five minutes.